

Rapport complet du travail de déploiement — Pi-CloudDOOM / fedipi

1. Objectif général du travail

L'objectif principal de notre travail était de déployer l'application **Pi-CloudDOOM** dans un environnement cloud réel, en partant d'une infrastructure OpenStack, puis en construisant un cluster Kubernetes capable d'exécuter les microservices de l'application.

Le travail ne s'est pas limité à lancer les services. Il a inclus :

- la préparation de l'infrastructure OpenStack ;
- la création ou recréation des machines virtuelles ;
- la correction des problèmes de boot, disques, volumes et snapshots ;
- la mise en place d'un cluster Kubernetes ;
- la correction des problèmes réseau, stockage et scheduling ;
- le déploiement progressif des microservices ;
- l'intégration des services externes comme Keycloak, OAuth Google, LinkedIn et GitHub ;
- l'exposition de l'application via Cloudflare ;
- la mise en place de la surveillance ;
- la préparation du module de détection d'attaques ;
- la documentation des erreurs rencontrées et des méthodes de résolution.

Le but final était d'obtenir un déploiement compréhensible, démontrable et défendable devant un encadrant ou un jury, en montrant non seulement le résultat final, mais aussi toutes les étapes techniques suivies.

2. Contexte de l'application

L'application concernée est **Pi-CloudDOOM**, une application microservices développée principalement avec :

- **Spring Boot** pour les services backend ;
- **Angular** pour le frontend ;
- **PostgreSQL** pour la base de données ;
- **Keycloak** pour l'authentification et la gestion des rôles ;
- **Kafka** pour la communication événementielle ;
- **Redis** pour le cache ;
- **Docker** pour la conteneurisation ;
- **Kubernetes** pour l'orchestration ;
- **OpenStack** comme plateforme cloud privée ;
- **Cloudflare Tunnel** pour exposer l'application vers Internet ;
- des composants de monitoring et de détection d'attaques pour renforcer la visibilité et la sécurité.

L'architecture cible est une architecture cloud-native basée sur plusieurs microservices. Chaque service doit être conteneurisé, poussé vers un registre Docker, puis déployé dans Kubernetes sous forme de Deployments, Services, ConfigMaps et Secrets.

3. Phase initiale : préparation de l'infrastructure OpenStack

3.1 Objectif de cette phase

La première étape a été de préparer l'environnement OpenStack qui allait héberger les machines virtuelles du cluster Kubernetes.

L'objectif était d'utiliser les ressources OpenStack disponibles pour créer plusieurs instances, puis de les transformer en nœuds Kubernetes.

L'environnement OpenStack comprenait plusieurs nœuds physiques ou compute nodes, notamment :

- `compute1yass` ;
- `compute2nacef` ;
- `compute3ayoub` ;
- des contrôleurs comme `ctrl-node` et `controller1aziz` ;
- un nœud de stockage référencé comme `storage1yosr`.

Nous avons aussi manipulé des composants OpenStack comme :

- Nova pour les instances ;
- Cinder pour les volumes ;
- Neutron pour le réseau ;
- Heat pour l'orchestration de stacks ;
- MariaDB/Galera pour le backend de la plateforme OpenStack.

3.2 Vérification des ressources disponibles

Avant de créer le cluster, il fallait vérifier les ressources disponibles sur les compute nodes.

Nous avons essayé d'utiliser :

```
openstack hypervisor show compute2nacef -c vcpus_used -c memory_mb_used -c running_vms
```

Mais cette commande n'a pas fonctionné, car certaines colonnes n'étaient pas reconnues par la version du client OpenStack utilisée. Le retour indiquait que les colonnes disponibles étaient différentes, par exemple :

- `aggregates` ;
- `cpu_info` ;

- `host_ip` ;
- `hypervisor_hostname` ;
- `state` ;
- `status` ;
- `uptime` ;
- `users` .

Cela a montré qu'il fallait adapter les commandes à la version exacte du client OpenStack et aux colonnes réellement exposées.

3.3 Utilisation de Heat

Nous avons travaillé avec Heat pour créer ou recréer une stack complète contenant les instances Kubernetes.

Le but était de pouvoir déployer le cluster de manière plus reproductible au lieu de créer chaque instance manuellement.

Nous avons rencontré des différences de syntaxe avec certaines commandes, notamment autour de :

```
openstack stack validate -t template.yaml
```

Certaines commandes ne fonctionnaient pas exactement comme prévu selon la version de l'outil. Nous avons donc utilisé d'autres commandes disponibles, comme :

```
openstack stack create
openstack stack update
openstack stack list
openstack stack show
openstack stack template show
```

L'objectif était de valider progressivement le template, lancer la stack, inspecter les erreurs et corriger les ressources.

4. Gestion des volumes, snapshots et erreurs de disque

4.1 Problème de volume à supprimer

Pendant le travail, nous avons voulu supprimer un volume OpenStack identifié par :

```
VOL=00404d60-d3e2-478d-81b0-aa7ff34f763d
```

Nous avons vérifié son état avec :

```
openstack volume show "$VOL" -c status -c attachments
openstack volume snapshot list --volume "$VOL"
```

Le volume était en état :

```
status: available
attachments: []
```

Cela signifie qu'il n'était attaché à aucune instance. Cependant, il possédait un snapshot :

```
63ca9e36-d46c-4858-b6c5-9c448571785e | k8s-cp1-root-before-fstab-fix | available
| 20G
```

La présence du snapshot empêchait ou compliquait la suppression propre du volume. La logique correcte est :

1. vérifier si le volume est attaché ;
2. détacher le volume si nécessaire ;
3. lister les snapshots associés ;
4. supprimer les snapshots ;
5. supprimer ensuite le volume.

Commandes typiques :

```
openstack volume snapshot delete 63ca9e36-d46c-4858-b6c5-9c448571785e
openstack volume delete 00404d60-d3e2-478d-81b0-aa7ff34f763d
```

4.2 Problème de boot sur `k8s-cp1`

Un problème important est apparu sur la machine `k8s-cp1`, notamment un blocage au démarrage lié au disque ou au système de fichiers.

Le contexte indiquait que la VM ne démarrait pas correctement et pouvait tomber dans un état de type emergency mode ou initramfs. Nous avons alors investigué :

- les volumes attachés ;
- le fichier `/etc/fstab` ;
- la possibilité qu'un volume attendu ne soit plus présent ;
- la nécessité de corriger le montage automatique ;
- la possibilité de restaurer ou manipuler un snapshot avant correction.

Le snapshot `k8s-cp1-root-before-fstab-fix` a été créé pour garder une sauvegarde avant modification critique.

La logique suivie était prudente : ne pas modifier directement le disque sans point de retour.

5. Recréation complète du cluster

5.1 Décision de repartir de zéro

Après plusieurs problèmes accumulés, nous avons décidé de supprimer l'ancien environnement et de **relancer une nouvelle stack depuis zéro**.

L'objectif était de repartir sur une base propre :

- anciennes instances supprimées ;
- volumes problématiques nettoyés ;
- stack Heat relancée ;
- cluster Kubernetes recréé proprement ;
- déploiement refait étape par étape.

Cette décision était importante, car elle évitait de continuer à corriger un environnement devenu instable.

5.2 Nouvelle architecture Kubernetes visée

Le cluster cible comportait plusieurs nœuds, avec des noms comme :

- `k8s-cp1` : control plane ;
- `k8s-w1` : worker ;
- `k8s-w2` : worker ;
- `k8s-w3` : worker ;
- `k8s-w4` : worker.

Le but était d'avoir un cluster Kubernetes capable d'héberger les microservices de l'application, avec un control plane et plusieurs workers pour répartir les pods.

6. Mise en place de Kubernetes

6.1 Vérification des nœuds

Après la création des instances, nous avons vérifié l'état du cluster avec :

```
kubectl get nodes -o wide
```

L'objectif était de confirmer que tous les nœuds étaient en état `Ready`.

Un cluster Kubernetes sain doit afficher :

NAME	STATUS	ROLES	AGE	VERSION
k8s-cp1	Ready	control-plane	...	...
k8s-w1	Ready	<none>	...	...
k8s-w2	Ready	<none>	...	...
k8s-w3	Ready	<none>	...	...
k8s-w4	Ready	<none>	...	...

6.2 Réseau Kubernetes

Pour que les pods puissent communiquer entre eux, il fallait installer un plugin CNI. Le choix mentionné était **Calico**.

Le rôle du CNI est essentiel : sans CNI fonctionnel, les pods restent souvent bloqués en `ContainerCreating`, ou les communications inter-pods échouent.

Les vérifications habituelles sont :

```
kubectl get pods -n kube-system -o wide
kubectl get nodes -o wide
```

Il faut vérifier que les pods Calico sont en état `Running`.

7. Problèmes liés au stockage Kubernetes

7.1 Absence de StorageClass

Pendant les tests, nous avons constaté que Kubernetes ne possédait pas de StorageClass disponible :

```
kubectl get storageclass
```

Résultat :

```
No resources found
```

Cela posait problème pour les workloads nécessitant des PersistentVolumeClaims, par exemple :

- PostgreSQL ;
- Keycloak si persistance nécessaire ;
- Prometheus ;
- Grafana ;
- certains services avec stockage local.

7.2 Installation de local-path-provisioner

Pour résoudre rapidement le problème de stockage dans un environnement de démonstration, nous avons installé **local-path-provisioner** :

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml
kubectl get storageclass
```

Les ressources créées incluaient :

- namespace `local-path-storage` ;
- ServiceAccount ;
- Role ;
- ClusterRole ;
- RoleBinding ;
- ClusterRoleBinding ;
- provisioner.

Ce choix est adapté à un environnement de test ou de démonstration, car il permet de créer automatiquement des volumes locaux sur les nœuds.

Limite importante : local-path-provisioner n'est pas équivalent à un vrai stockage distribué hautement disponible. Si le pod est reschedulé sur un autre nœud, les données peuvent ne pas être disponibles sur ce nouveau nœud.

8. Tests initiaux dans Kubernetes

8.1 Test avec Nginx

Avant de déployer les microservices réels, nous avons utilisé un déploiement simple comme Nginx pour vérifier que Kubernetes fonctionnait correctement.

Commandes utilisées ou équivalentes :

```
kubectl get pods -o wide -w  
kubectl delete deployment nginx-test
```

Le pod Nginx est passé en état `Running`, ce qui a confirmé que :

- le scheduler fonctionnait ;
- les workers pouvaient exécuter des pods ;
- le runtime conteneur fonctionnait ;
- le réseau pod de base était opérationnel.

9. Namespace applicatif

Pour organiser les ressources de l'application, nous avons utilisé le namespace :

```
piclouddoom
```

Les commandes de vérification étaient souvent exécutées avec :

```
kubectl get pods -n piclouddoom -o wide  
kubectl get deploy -n piclouddoom  
kubectl get svc -n piclouddoom
```

Le namespace permet de séparer les ressources applicatives des ressources système comme `kube-system`, `local-path-storage`, `monitoring`, etc.

10. Déploiement des microservices

10.1 Services visibles dans le cluster

Nous avons travaillé sur plusieurs services de Pi-CloudDOOM, notamment :

- `user-service` ;
- `community-service` ;
- `interview-service` ;
- possiblement d'autres services comme training, notification, gateway ou frontend selon l'état du projet ;
- des services d'infrastructure comme PostgreSQL, Redis, Kafka et Keycloak.

Un exemple d'état observé :


```
community-service-7bbd7c9c6b-x2rgl    1/1    Running
interview-service-b4bfbfddf-hsv9l     1/1    Running
user-service                           0/1    problème d'image ou de readiness
```

Cela montrait que certains services fonctionnaient, tandis que `user-service` nécessitait une correction spécifique.

10.2 Vérification générale des pods

Commande principale :

```
kubectl -n piclouddoom get pods -o wide
```

Cette commande permettait de voir :

- le nom du pod ;
- son état ;
- le nombre de restarts ;
- son âge ;
- son IP interne ;
- le nœud sur lequel il tourne.

Exemple de pod sain :

```
community-service-7bbd7c9c6b-x2rgl    1/1    Running    2    ...    k8s-w4
```

Exemple de pod problématique :

```
user-service-5fcbf5fbb5-mkq44    0/1    ErrImagePull
```

11. Problème du `user-service`

11.1 Symptôme principal

Le deployment `user-service` existait, mais il n'était pas disponible :

```
kubectl get deploy user-service -n piclouddoom -o wide
```

Résultat observé :

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES
user-service	0/1	1	0	27h	user-service	docker.io/ azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-v2

Le pod associé affichait :

```
ErrImagePull
```

11.2 Cause probable

L'image utilisée était :

```
docker.io/azizbna/pi-clouddoom-user-service:cv-ollama-fast-v2-v2
```

Un état `ErrImagePull` signifie généralement que Kubernetes n'arrive pas à récupérer l'image depuis le registre Docker.

Causes possibles :

- tag inexistant ;
- image non poussée vers Docker Hub ;
- repository privé ;
- mauvaise orthographe du nom d'image ;
- problème d'accès Internet depuis le nœud ;
- problème de credentials Docker ;
- architecture incompatible de l'image.

11.3 Diagnostic recommandé

Les commandes essentielles sont :

```
kubectl describe pod <nom-du-pod> -n piclouddoom
kubectl get events -n piclouddoom --sort-by=.lastTimestamp
kubectl get deploy user-service -n piclouddoom -o yaml
kubectl get deploy user-service -n piclouddoom -o
jsonpath='{.spec.template.spec.containers[0].image}'{"\n"}
```

Elles permettent de confirmer si le problème vient bien du pull de l'image.

11.4 Rollback tenté

Nous avons tenté un rollback du deployment :

```
kubectl rollout undo deployment/user-service -n piclouddoom
kubectl rollout status deployment/user-service -n piclouddoom --timeout=180s
```

Mais le rollout a expiré :

```
error: timed out waiting for the condition
```

Cela signifie que même après rollback, Kubernetes n'a pas réussi à rendre le pod disponible.

Le message :

```
Waiting for deployment "user-service" rollout to finish: 0 of 1 updated replicas
are available...
```

montre que le pod n'arrivait pas à atteindre l'état `Ready`.

11.5 Méthode correcte pour corriger

La méthode recommandée est :

1. identifier l'image actuellement utilisée ;
2. vérifier que le tag existe réellement dans Docker Hub ;
3. pousser l'image si elle n'existe pas ;
4. ou remplacer l'image par un tag valide ;
5. relancer le rollout.

Exemple :

```
kubectl -n piclouddoom set image deployment/user-service
user-service=docker.io/azizbna/pi-clouddoom-user-service:<tag-valide>

kubectl -n piclouddoom rollout status deployment/user-service --timeout=180s
kubectl -n piclouddoom get pods -l app=user-service -o wide
```

Si le repository est privé, il faut créer un `imagePullSecret` :

```
kubectl create secret docker-registry dockerhub-secret
--docker-server=https://index.docker.io/v1/
--docker-username=<username>
--docker-password=<password>
```

```
--docker-email=<email>  
-n piclouddoom
```

Puis l'associer au deployment ou au service account.

12. Authentification et Keycloak

12.1 Rôle de Keycloak

Keycloak est utilisé pour gérer :

- l'authentification ;
- les utilisateurs ;
- les rôles ;
- le SSO ;
- les fournisseurs externes comme Google, LinkedIn et GitHub.

Le realm utilisé dans le projet est :

```
myapp-realm
```

L'issuer URI typique côté Spring Boot est :

```
http://localhost:8080/realms/myapp-realm
```

Dans Kubernetes, cette URL doit être adaptée selon le service Keycloak interne ou l'URL publique exposée.

12.2 Rôles Keycloak

Les rôles utilisés côté Keycloak incluent :

- `ROLE_USER` ;
- `ROLE_ADMIN` ;
- `ROLE_MANAGER` .

Côté Spring Security, les rôles du token JWT sont lus depuis :

```
realm_access.roles
```

Puis transformés en autorités Spring Security via un convertisseur JWT.

Il est important de ne pas confondre :

- les rôles applicatifs ou métier comme `FREE`, `PREMIUM`, `ADMIN` ;
- les rôles de sécurité Keycloak comme `ROLE_USER`, `ROLE_ADMIN`, `ROLE_MANAGER` .

12.3 Problèmes OAuth Google, LinkedIn et GitHub

Nous avons travaillé sur les connexions sociales :

- LinkedIn ;
- Google ;
- GitHub.

LinkedIn a été corrigé, mais Google restait problématique pendant une étape. Ensuite, nous avons déplacé le diagnostic vers GitHub.

Les symptômes observés incluaient :

- erreur après clic sur "Continue with Google" ;
- erreur après clic sur "Continue with GitHub" ;
- problème de redirect URI ;
- problème de configuration du provider dans Keycloak ;
- différence entre l'URL publique Cloudflare et l'URL interne Kubernetes ;
- mismatch entre l'URL configurée dans Google/GitHub et celle réellement appelée par Keycloak.

La logique de correction était :

1. ouvrir la configuration du provider dans Keycloak ;
2. récupérer l'URL de redirection générée par Keycloak ;
3. copier exactement cette URL dans Google Cloud Console ou GitHub OAuth App ;
4. vérifier le client ID ;
5. vérifier le client secret ;
6. vérifier que le realm et le client Angular utilisent les bonnes URLs ;
7. tester à nouveau depuis le frontend.

12.4 Point important sur Cloudflare et OAuth

Quand l'application est exposée via Cloudflare, l'URL publique devient l'URL que les providers OAuth doivent connaître.

Exemple conceptuel :

```
https://<domaine-public>/realms/myapp-realm/broker/google/endpoint
```

ou :

```
https://<domaine-public>/realms/myapp-realm/broker/github/endpoint
```

Il ne faut pas utiliser une URL interne Kubernetes ou localhost dans la configuration Google/GitHub quand l'utilisateur se connecte depuis Internet.

13. Cloudflare Tunnel et erreur 1033

13.1 Symptôme

Après un reboot, une erreur Cloudflare est apparue :

```
Cloudflare Error 1033
```

Cette erreur signifie généralement que Cloudflare ne parvient pas à atteindre le tunnel ou l'origine associée.

13.2 Cause probable

Après un reboot, le tunnel Cloudflare peut ne pas être relancé automatiquement, ou bien le service local qu'il expose peut ne pas être disponible.

Causes possibles :

- `cloudflared` arrêté ;
- service Kubernetes non prêt ;
- ingress ou port local non disponible ;
- tunnel non connecté ;
- DNS Cloudflare pointant vers un tunnel inactif.

13.3 Diagnostic

Commandes utiles :

```
systemctl status cloudflared  
journalctl -u cloudflared -f  
cloudflared tunnel list  
cloudflared tunnel info <tunnel-name>
```

Côté Kubernetes :

```
kubectl get pods -A
kubectl get svc -A
kubectl get ingress -A
```

13.4 Correction

La correction générale consiste à :

```
sudo systemctl restart cloudflared
sudo systemctl enable cloudflared
```

Puis vérifier que le service cible fonctionne réellement.

14. Problèmes SSH et host key

14.1 Symptôme

Après recréation d'instances, SSH a affiché :

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
```

Exemple :

```
ssh -i ~/heat-eval-key.pem ubuntu@192.168.1.209
```

Le message indiquait que la clé SSH de l'hôte avait changé.

14.2 Cause

Ce problème arrive souvent quand une instance est supprimée puis recréée avec la même adresse IP. La machine est nouvelle, donc sa clé SSH est différente, mais le fichier local `known_hosts` contient encore l'ancienne clé.

14.3 Correction

La commande suggérée était :

```
ssh-keygen -f '/home/ctrl/.ssh/known_hosts' -R '192.168.1.209'
```

Puis reconnexion :

```
ssh -i ~/heat-eval-key.pem ubuntu@192.168.1.209
```

Il faut accepter la nouvelle empreinte uniquement si on sait que l'instance a bien été recréée par nous.

15. MariaDB/Galera côté OpenStack

15.1 Contexte

Pendant la préparation OpenStack, nous avons aussi rencontré des problèmes liés au cluster MariaDB/Galera sur les contrôleurs.

Les nœuds concernés incluaient :

- `ctrl-node` ;
- `controller1aziz` .

Le cluster Galera était nommé :

```
openstack-galera-cluster
```

15.2 Problèmes observés

Nous avons vu des situations où :

- un nœud Galera ne communiquait plus avec l'autre ;
- les services OpenStack dépendants de MariaDB étaient impactés ;
- les services Cinder pouvaient apparaître `down` ;
- les variables d'environnement liées à WSREP pouvaient poser problème après reboot.

Exemple de diagnostic :

```
sudo rabbitmqctl cluster_status
```

Même si cette commande concerne RabbitMQ, elle a aidé à comprendre les problèmes de communication entre contrôleurs dans l'environnement OpenStack.

Pour Galera, les vérifications typiques étaient :

```
sudo systemctl status mariadb
sudo journalctl -u mariadb -xe
sudo grep -R "wsrep_cluster_address" /etc/mysql/ /etc/my.cnf* 2>/dev/null
```



```
sudo grep -R "wsrep_node_address" /etc/mysql/ /etc/my.cnf* 2>/dev/null  
sudo ss -lntp | egrep '3306|4444|4567|4568'
```

15.3 Importance pour le déploiement

Même si Galera n'est pas directement dans Kubernetes, il est critique pour OpenStack. Si OpenStack est instable, la création des instances, volumes ou réseaux peut échouer.

16. RabbitMQ côté OpenStack

16.1 Diagnostic du cluster RabbitMQ

Nous avons aussi vérifié RabbitMQ avec :

```
sudo rabbitmqctl cluster_status
```

Le résultat montrait notamment :

```
Running Nodes  
rabbit@ctrl-node  
  
Network Partitions  
Node rabbit@ctrl-node cannot communicate with rabbit@controller1aziz
```

Cela indiquait un problème de communication entre les nœuds RabbitMQ.

16.2 Impact

RabbitMQ est utilisé par plusieurs services OpenStack pour la communication interne. Si RabbitMQ est cassé ou partitionné, OpenStack peut devenir instable.

Cela peut impacter :

- Nova ;
- Cinder ;
- Neutron ;
- les opérations de création ou suppression d'instances ;
- les volumes ;
- les états de services.

17. Monitoring

17.1 Objectif du monitoring

Le monitoring est nécessaire pour observer l'état du cluster et détecter les anomalies.

Il permet de surveiller :

- l'état des nœuds Kubernetes ;
- l'utilisation CPU ;
- l'utilisation mémoire ;
- l'état des pods ;
- les restarts ;
- les erreurs applicatives ;
- les métriques HTTP ;
- les métriques système ;
- les alertes de sécurité.

17.2 Stack de monitoring recommandée

La stack logique pour Kubernetes est :

- Prometheus pour collecter les métriques ;
- Grafana pour visualiser les dashboards ;
- Alertmanager pour gérer les alertes ;
- Node Exporter pour les métriques des nœuds ;
- kube-state-metrics pour les objets Kubernetes.

Une installation typique peut se faire via Helm :

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack
-n monitoring --create-namespace
```

Vérification :

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring
```

17.3 Exposition de Grafana

Pour accéder à Grafana :

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

Puis ouvrir :

```
http://localhost:3000
```

Dans un environnement exposé publiquement, il faut sécuriser Grafana avec :

- authentification forte ;
- mot de passe modifié ;
- accès limité ;
- éventuellement exposition via Cloudflare avec règles d'accès.

17.4 Métriques importantes à présenter

Pour une démonstration, les métriques les plus importantes sont :

- état des nœuds ;
- nombre de pods Running/Pending/CrashLoopBackOff ;
- CPU par namespace ;
- mémoire par namespace ;
- restarts des pods ;
- latence HTTP des services ;
- taux d'erreurs HTTP 4xx/5xx ;
- état des PersistentVolumeClaims ;
- disponibilité du `user-service`, `community-service`, `interview-service`.

18. Attack Detector

18.1 Objectif

Le module d'attaque detector vise à identifier des comportements suspects dans l'environnement applicatif ou Kubernetes.

Il peut détecter par exemple :

- trop de requêtes en peu de temps ;
- tentatives répétées de login ;
- erreurs HTTP répétées ;
- accès anormaux aux endpoints sensibles ;
- comportement ressemblant à du brute force ;
- scans d'URLs ;
- pics d'erreurs applicatives ;

- augmentation inhabituelle des restarts de pods.

18.2 Sources de données possibles

Le détecteur peut s'appuyer sur :

- logs des pods ;
- logs Ingress ;
- métriques Prometheus ;
- événements Kubernetes ;
- logs applicatifs Spring Boot ;
- logs Keycloak ;
- logs Cloudflare ;
- logs système des nœuds.

Commandes utiles :

```
kubectl logs -n piclouddoom deployment/user-service
kubectl logs -n piclouddoom deployment/community-service
kubectl logs -n piclouddoom deployment/interview-service
kubectl get events -n piclouddoom --sort-by=.lastTimestamp
```

18.3 Architecture logique

L'architecture du module peut être décrite comme suit :

```
Applications / Ingress / Keycloak / Kubernetes Events
      ↓
    Logs et métriques
      ↓
    Collecte / parsing
      ↓
    Règles de détection
      ↓
    Alertes / dashboard
```

18.4 Règles simples de détection

Exemples de règles :

```
Si nombre de réponses HTTP 401 depuis la même IP > seuil pendant 5 minutes
=> suspicion de brute force
```

Si nombre de requêtes vers des routes inexistantes > seuil
=> suspicion de scan

Si un pod redémarre plusieurs fois en peu de temps
=> suspicion de problème applicatif ou attaque provoquant une instabilité

Si Keycloak reçoit beaucoup d'échecs d'authentification
=> suspicion d'attaque sur login

18.5 Intégration avec Prometheus

Si les métriques sont disponibles dans Prometheus, on peut créer des alertes PromQL.

Exemple conceptuel :

```
increase(http_server_requests_seconds_count{status="401"}[5m]) > 20
```

Ou pour les restarts Kubernetes :

```
increase(kube_pod_container_status_restarts_total[10m]) > 3
```

Ces alertes peuvent ensuite être visualisées dans Grafana ou envoyées vers Alertmanager.

19. Gestion des images Docker

19.1 Importance des tags

Un problème majeur rencontré avec `user-service` était lié à l'image Docker. Cela montre l'importance d'une stratégie claire de tags.

Mauvaise pratique :

```
latest
```

ou tags créés sans certitude qu'ils existent.

Meilleure pratique :

```
pi-clouddoom-user-service:v1.0.0
pi-clouddoom-user-service:cv-ollama-fast-v2
pi-clouddoom-user-service:demo-stable
```

Chaque tag utilisé par Kubernetes doit exister dans Docker Hub.

19.2 Vérification côté Kubernetes

```
kubectl get deploy user-service -n piclouddoom -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

19.3 Correction d'image

```
kubectl -n piclouddoom set image deployment/user-service
user-service=docker.io/azizbna/pi-clouddoom-user-service:<tag-correct>
```

Puis :

```
kubectl -n piclouddoom rollout status deployment/user-service
```

20. Méthode générale de troubleshooting Kubernetes utilisée

Pendant tout le travail, nous avons suivi une méthode progressive :

20.1 Voir l'état global

```
kubectl get pods -A -o wide
kubectl get nodes -o wide
kubectl get svc -A
```

20.2 Diagnostiquer un pod précis

```
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
```

20.3 Voir les événements

```
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

20.4 Vérifier le deployment

```
kubectl get deploy <deployment-name> -n <namespace> -o yaml
kubectl rollout status deployment/<deployment-name> -n <namespace>
kubectl rollout history deployment/<deployment-name> -n <namespace>
```

20.5 Rollback

```
kubectl rollout undo deployment/<deployment-name> -n <namespace>
```

20.6 Redémarrage contrôlé

```
kubectl rollout restart deployment/<deployment-name> -n <namespace>
```

21. Erreurs importantes rencontrées et corrections

Problème	Symptôme	Cause probable	Correction
StorageClass absente	No resources found	Aucun provisioner installé	Installation de local-path-provisioner
Pod <code>user-service</code> non disponible	ErrImagePull	Image Docker/tag introuvable ou privé	Vérifier/pousser l'image ou corriger le tag
Rollout bloqué	timed out waiting for the condition	Pod non Ready	<code>describe</code> , logs, events, image fix
Cloudflare 1033	Site inaccessible après reboot	Tunnel ou origine indisponible	Redémarrer/activer <code>cloudflared</code> , vérifier services
SSH host key changed	Warning SSH	VM recrée avec même IP	<code>ssh-keygen -R <IP></code>
OAuth Google/GitHub	Erreur après login	Redirect URI incorrecte	Copier l'URL exacte Keycloak dans provider OAuth

Problème	Symptôme	Cause probable	Correction
VM bloquée au boot	emergency/initramfs	disque/fstab/ volume	Corriger fstab, vérifier volumes, snapshot avant modif
OpenStack instable	services down	Galera/RabbitMQ réseau	vérifier cluster, ports, systemd, logs

22. Résultat obtenu progressivement

À travers ces étapes, nous avons réussi à :

- préparer l'environnement OpenStack ;
- comprendre et corriger plusieurs problèmes d'infrastructure ;
- recréer un cluster Kubernetes propre ;
- vérifier le fonctionnement des nœuds ;
- ajouter une solution de stockage avec local-path-provisioner ;
- tester Kubernetes avec un pod simple ;
- déployer plusieurs services applicatifs ;
- identifier les services fonctionnels ;
- diagnostiquer le problème du `user-service` ;
- travailler sur les connexions OAuth ;
- corriger LinkedIn ;
- investiguer Google puis GitHub ;
- exposer l'application via Cloudflare ;
- identifier l'erreur 1033 après reboot ;
- définir une stratégie de monitoring ;
- préparer la logique du module attack detector.

23. Architecture finale attendue

L'architecture finale peut être décrite ainsi :

```

Utilisateur
↓
Cloudflare Tunnel / Domaine public
↓
Ingress ou Service Kubernetes
↓
Frontend Angular
↓
Microservices Spring Boot

```



```
↓  
PostgreSQL / Redis / Kafka / Keycloak  
↓  
Monitoring + Attack Detector
```

Et côté infrastructure :

```
OpenStack  
↓  
Instances Ubuntu  
↓  
Cluster Kubernetes  
↓  
Namespaces applicatifs  
↓  
Deployments / Services / PVC / ConfigMaps / Secrets
```

24. Bonnes pratiques retenues

24.1 Infrastructure

- Toujours vérifier les ressources OpenStack avant de déployer.
- Nettoyer les anciens volumes et snapshots avant de recréer une stack.
- Garder un snapshot avant toute correction critique du disque.
- Vérifier l'état des services OpenStack après reboot.

24.2 Kubernetes

- Toujours commencer par `kubectl get pods -A`.
- Vérifier les events avant de modifier au hasard.
- Installer une StorageClass avant de déployer des services persistants.
- Ne pas utiliser de tag Docker incertain dans un deployment.
- Utiliser un namespace séparé pour l'application.

24.3 Sécurité

- Ne pas exposer Keycloak avec des URLs mal configurées.
- Toujours aligner les redirect URIs OAuth avec l'URL publique réelle.
- Changer les mots de passe par défaut de Grafana et Keycloak.
- Stocker les secrets dans Kubernetes Secrets, pas en clair dans les manifests.

24.4 Démonstration

Pour une démonstration devant un jury, il faut montrer :

1. l'application accessible via le domaine public ;
2. les pods Kubernetes en état `Running` ;
3. les services exposés ;
4. Keycloak et le login social ;
5. Grafana avec les métriques ;
6. un exemple d'alerte ou détection d'anomalie ;
7. la capacité à expliquer les erreurs rencontrées et les corrections.

25. Commandes principales à retenir

OpenStack

```
source ~/admin-openrc
openstack stack list
openstack stack show <stack-name>
openstack stack create -t <template.yaml> <stack-name>
openstack stack delete <stack-name>
openstack volume list
openstack volume show <volume-id>
openstack volume snapshot list --volume <volume-id>
openstack server list
```

SSH

```
ssh -i ~/heat-eval-key.pem ubuntu@<ip>
ssh-keygen -f ~/.ssh/known_hosts -R <ip>
```

Kubernetes global

```
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get svc -A
kubectl get events -A --sort-by=.lastTimestamp
```

Namespace application

```
kubectl get pods -n piclouddoom -o wide
kubectl get deploy -n piclouddoom
```

```
kubectl get svc -n piclouddoom
kubectl get events -n piclouddoom --sort-by=.lastTimestamp
```

Debug pod

```
kubectl describe pod <pod-name> -n piclouddoom
kubectl logs <pod-name> -n piclouddoom
```

Debug deployment

```
kubectl rollout status deployment/<name> -n piclouddoom
kubectl rollout history deployment/<name> -n piclouddoom
kubectl rollout undo deployment/<name> -n piclouddoom
kubectl rollout restart deployment/<name> -n piclouddoom
```

Image d'un deployment

```
kubectl get deploy user-service -n piclouddoom
-o jsonpath='{.spec.template.spec.containers[0].image}'
```

Modifier image

```
kubectl -n piclouddoom set image deployment/user-service
user-service=docker.io/azizbna/pi-clouddoom-user-service:<tag>
```

Monitoring

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-
charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack
-n monitoring --create-namespace
kubectl get pods -n monitoring
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

26. Conclusion

Le travail réalisé représente un cycle complet de déploiement cloud-native : nous sommes partis d'une infrastructure OpenStack, nous avons traité les problèmes bas niveau liés aux volumes, snapshots, SSH,

services OpenStack et reboot, puis nous avons construit un cluster Kubernetes capable d'héberger l'application Pi-CloudDOOM.

Ensuite, nous avons déployé progressivement les microservices, corrigé les problèmes de stockage, d'images Docker, de rollout Kubernetes, d'exposition Cloudflare et d'authentification OAuth.

La partie la plus importante du travail n'est pas seulement le fait de lancer les pods, mais la capacité à diagnostiquer chaque problème avec méthode : regarder les états, lire les events, inspecter les logs, vérifier les images, corriger les URLs, relancer proprement et documenter chaque correction.

Le projet final montre donc une maîtrise pratique de :

- OpenStack ;
- Kubernetes ;
- Docker ;
- microservices Spring Boot ;
- Keycloak et OAuth ;
- Cloudflare Tunnel ;
- monitoring avec Prometheus/Grafana ;
- détection d'anomalies et sécurité applicative ;
- troubleshooting réel en environnement cloud.

Ce rapport peut servir de base pour la soutenance, la documentation technique ou la rédaction finale du projet.